Cloud Computing (CS3081) — Comprehensive Project Reflection

Student Name: Yograj Chahar | **Roll Number:** [Roll Number] | **Repository:** github.com/yograjchahar05/cc-assignment | **Release:** v1.0.0

1. Knowledge Distillation Under Practical Constraints

The objective of transferring natural language parsing capabilities from a 3.8-billion parameter teacher model (Phi-3.5) into a compact 70-million parameter student (Pythia-70M) provided profound practical insights into the realities of cloud-native machine learning. Before this project, I viewed distillation primarily through the lens of academic literature—specifically logit-based matching using Kullback-Leibler divergence over soft softmax distributions. However, confronting real-world engineering constraints immediately revealed why logit distillation is frequently impossible in production architectures:

- *Tokenizer and Vocabulary Incommensurability:* Phi-3.5 relies on a SentencePiece tokenizer with 32,064 tokens, while Pythia-70M utilizes a GPT-NeoX byte-pair encoding (BPE) vocabulary of 50,304 tokens. The token boundaries, subword segmentations, and semantic coordinate spaces differ entirely. An element-by-element loss calculation between the output probability vectors is mathematically undefined without introducing an auxiliary projection matrix that would distort the teacher's representations.
- *Black-Box API Isolation:* Deploying the teacher via Ollama encapsulates the neural network behind an HTTP REST interface. Production inference engines rarely expose internal activation layers or multi-gigabyte unnormalized logit tensors due to bandwidth overhead and security isolation. Sequence-level distillation—treating teacher completions as pseudo-ground-truth targets—proved to be the only viable paradigm.

2. Deep Debugging: CPU Floating-Point Overflow & Gradient Instability

One of the most demanding engineering challenges encountered during Task 2 was an abrupt training failure where PyTorch loss collapsed to NaN within 20 gradient steps. Initially, I suspected an excessively aggressive learning rate or exploding gradients. However, gradient clipping at max_norm=1.0 did not resolve the issue.

By instrumenting individual weight tensors and AdamW optimizer buffers, I isolated the root cause: **FP16 uncentered second-moment overflow on CPU**. Pythia-70M weights on HuggingFace default to half-precision (torch.float16). Unlike modern server GPUs with specialized FP16 tensor cores, x86-64 CPUs lack native IEEE 754 half-precision arithmetic units. In FP16, the maximum representable finite number is 65,504. In AdamW, the uncentered variance accumulator $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ rapidly exceeded 65,504 when unscaled gradients were squared, instantly producing inf and propagating NaN across all subsequent weight updates.

The Solution: Forcing model loading in full 32-bit floating point precision (dtype=torch.float32) completely eliminated numerical instability. Furthermore, by replacing static max-length sequence padding with dynamic batch padding in a custom collate_fn, padding tokens were reduced by 78%, accelerating CPU batch throughput by 4.5× and enabling 3 complete training epochs in under 3 minutes.

3. Twelve-Factor Cloud Architecture & Microservice Decoupling

Designing the FastAPI inference microservice in Task 3 reinforced core tenets of modern cloud systems architecture. The temptation in ML engineering is to bake model weights directly into the container filesystem (COPY models/ /app/models). While straightforward, this violates the decoupling principle of cloud-native systems:

- *Image Bloat and CI/CD Inefficiency:* Embedding a 281 MB model into the image bloats image transfer times, inflates container registry storage costs, and couples code deployment with model lifecycle. Any hyperparameter retraining would necessitate a complete Docker build and push cycle.
- *Dynamic Object Storage Decoupling:* Implementing startup weight hydration from SeaweedFS S3 via boto3 decoupled code from state. The microservice image contains only runtime Python logic and pinned dependencies (<1.2 GB), allowing the same immutable image to serve version v1, v2, or future models simply by modifying the MODEL_URI environment variable.
- *Readiness Probe Gatekeeping:* Downloading 281 MB of weights over an internal network introduces a ~14-second cold-start window. If a pod accepted ingress traffic before weights finished downloading into RAM, incoming requests would suffer HTTP 500 errors. Implementing a rigorous readiness probe on /readyz (returning HTTP 503 until memory allocation completes) guaranteed that Kubernetes routing tables never directed traffic to unready pods, achieving a 100% success rate during rolling updates and pod restarts.

4. Network Security: Stateless Packet Filtering vs Stateful Firewalls

Task 5's requirement to implement a stateless Network Access Control List (NACL) for the public subnet highlighted fundamental differences between layer 3/4 packet filters and stateful security groups:

- *The Bidirectional Stateless Trap:* Developers accustomed to stateful firewalls often expect that permitting inbound TCP 80 automatically allows the server's reply. In a stateless NACL, the firewall has no connection memory (no state table tracking SYN/ACK handshakes). Every single outbound reply packet from the web server originates from port 80 and targets the client's randomly assigned ephemeral port (1024–65535). Without an explicit outbound rule permitting ephemeral return traffic, every outgoing response is silently dropped by the default deny rule.
- *Strict Rule Order Hierarchy:* A crucial architectural insight was the evaluation order of access control entries. Because NACLs evaluate rules sequentially and stop on first match, a generic ALLOW 0.0.0.0/0:80 rule placed before a specific DENY 198.51.100.0/24 rule would inadvertently allow traffic from the abusive range. Ordering the abusive range drop as Rule 50 ahead of general web rules (Rules 100/110) proved essential for robust defensive posture.

5. Kubernetes Operational Reliability and Laptop Constraints

Testing Kubernetes resiliency primitives (HPA autoscaling, zero-downtime rolling updates, and pod self-healing) provided practical validation of container orchestration semantics. During live Locust load testing, deleting an active replica resulted in zero dropped requests because the surviving replica absorbed the load while kubelet rescheduled the failed container within 1.2 seconds.

Key Observation on Local Clusters: Running k3d on a developer laptop revealed a critical architectural boundary: *software fault tolerance is not physical fault tolerance*. While Kubernetes gracefully handles container crashes and pod evictions, all nodes share the same physical RAM, CPU cores, thermal throttling constraints, and power supply. In production, true enterprise resilience demands multi-zone node groups with pod anti-affinity rules spanning distinct geographical availability zones.

6. Automated CI/CD as an Architectural Safety Gate

In traditional software engineering, continuous integration pipelines validate compilation and unit test assertions. In machine learning systems, code can be completely bug-free while the underlying model produces garbage predictions. Task 6 demonstrated the necessity of **ML Quality Gating**:

- *Automated Regression Prevention:* By embedding score.py --min-validity 95 directly into GitHub Actions after unit testing, the pipeline treats statistical regression with the same severity as a syntax error. When the pipeline was intentionally executed against the untrained baseline Pythia-70M model, the quality gate immediately rejected the 0% valid outputs, failing the build and preventing corrupted weights from ever reaching staging or production.
- *Supply-Chain Integrity:* The CI workflow downloads model assets from GitHub Release tags and verifies cryptographic SHA-256 checksums before evaluation. This ensures that the exact binary artifact trained locally is the one deployed, preventing intermediate tampering or bit rot.

7. Cloud Economics and SLO Error Budget Management

Translating technical throughput into financial and contractual metrics in Task 7 bridged systems engineering with business economics:

- *Unit Economics Analysis:* Evaluating the microservice using data/rate_card.json revealed that at ■3.95 per replica-hour and a sustained throughput of 2.38 requests/second (8,581.6 req/hr), the serving cost is **■0.4603 per 1,000 requests** (less than half a rupee). However, because Kubernetes replicas are billed for allocated limits regardless of utilization, traffic troughs represent pure financial waste. This highlights the economic justification for serverless container architectures (e.g. Knative or AWS Fargate) that scale to zero during idle periods.
- *SLO Error Budget as an Innovation Velocity Enabler:* Defining our Service Level Objective (99.0% success over 2 minutes) yielded an error budget of 2.84 allowable failures. Achieving 100.0% availability consumed 0% of the error budget. In an SRE context, this unused budget is not merely a safety cushion—it provides the operational license to perform rapid canary deployments, experiment with aggressive model quantizations, and iterate rapidly without breaching client service agreements.

8. Retrospective: What Could Have Been Done Better

Reflecting on the complete project lifecycle, several architectural enhancements could be pursued in future iterations:

1. **Inference Quantization & Acceleration:** PyTorch CPU inference on FP32 Pythia-70M resulted in average latencies of ~440ms. Converting the trained model to ONNX Runtime with INT8 dynamic quantization or using GGML (llama.cpp) would reduce memory bandwidth requirements and slash inference latency below 80ms on standard x86 CPU cores.
2. **Teacher Generation Throughput:** Labelling 3,000 commands using sequential Ollama REST calls was compute-intensive. Leveraging asynchronous batch token generation via vLLM or Hugging Face TGI with Tensor Parallelism would compress dataset creation time from hours to minutes.
3. **Student Capacity vs Context Reasoning:** While Pythia-70M achieved 100% JSON validity and excelled at verbatim slot copying (89% time accuracy), its intent accuracy (59%) reflects the parameter capacity limits of a 70M model on subtle linguistic disambiguation. Distilling into a modern 0.5B architecture (such as Qwen2.5-0.5B) would provide higher reasoning fidelity while remaining within the 1 vCPU / 1 GiB edge budget.